



UNIVERSIDADE D
COIMBRA

Classificação de Atividades Humanas ECAC 2025

Martim Alves Rodrigues da Costa Duarte,nº2021275991

Simão Tomás Botas Carvalho,nº2021223055

Índice

Índice.....	2
1. Resumo.....	3
2. Introdução.....	3
3. Explicação do código e comentários.....	4
3.1 e 3.2.....	4
3.3 e 3.4.....	6
3.5.....	8
3.6 e 3.7.....	9
3.7.1.....	11
4.1.....	13
4.2.....	14
4.3.....	15
4.4.1.....	15
4.4.2.....	15
4.5.....	15
4.6.....	16
4.6.1.....	16
4.6.2.....	16
Conclusão.....	16

1. Resumo

Este trabalho implementou uma pipeline completa de análise de dados para classificação de atividades humanas usando o dataset FORTH-TRACE. Foram aplicadas técnicas de detecção de outliers (**Z-Score** e **K-means** e **DBSCAN**), extração de features temporais e espectrais, redução de dimensionalidade com **PCA** e seleção de características com **Fisher Score** e **ReliefF**. Os resultados confirmaram a viabilidade de distinguir atividades através de sensores *wearable*, respondendo às questões do enunciado sobre preparação de dados, engenharia de características e análise estatística. O trabalho demonstrou a importância de cada fase do processo para o reconhecimento eficaz de atividades humanas.

2. Introdução

É importante referir que para o desenvolvimento deste trabalho foi usada a plataforma Git para controlo de versões e Gitlab para poder colaborar em conjunto com ajuda de um servidor remoto.

Cada função referenciada ao longo do relatório contém um *docstring* que apresenta uma visão geral da função assim como a explicação de todos os parâmetros de entrada e saída da função.

A pipeline de análise de dados desenvolvida neste trabalho foi feita de acordo com o documento fornecido, assim, começamos por importar o dataset. Para tal foi desenvolvida uma função *get_data* presente no ficheiro *data_treatment.py* (ponto 2).

Todo o trabalho foi devidamente estruturado com vários ficheiros *.py* para melhor organização do código.

2.1 Classificação das atividades

- Para reduzir a escrita e repetição de texto, assim como facilitar as nossas análises, as atividades foram divididas nos seguintes grupos:
-

Atividades estáticas:

1. Estar em pé
 2. Sentar
 3. Sentar e falar
-

Atividades dinâmicas:

4. Andar
 5. Andar e falar
 6. Subir/descer escadas
 7. Subir/descer escadas e falar
-

Atividades de transição:

8. Estar em pé → Sentar
 9. Sentar → Estar em pé
 10. Estar em pé → Sentar e falar
 11. Sentar → Estar em pé e falar
 12. Estar em pé → Andar
 13. Andar → Estar em pé
 14. Estar em pé → Subir/descer escadas / Estar em pé → Subir/descer escadas e falar
 15. Subir/descer escadas → Andar
 16. Subir/descer escadas e falar → Andar e falar
-

3. Explicação do código e comentários

3.1 e 3.2

Neste ponto foi criada uma função *calculate_module* que é integrada na função *calculate_modules* que calcula os módulos de todos os vetores (aceleração, giroscópio e magnetômetro).

Para os boxplots e densidade foi criado um ficheiro *boxplot.py* contendo uma função *create_boxplot_per_activity* que mostra os boxplots e calcula as densidades de outliers.

Aplicando o método IQR apenas ao sensor do pulso direito, observou-se padrões distintos na densidade de outliers entre sensores e atividades:

- Para a aceleração, as atividades de transição (A8-A12 e A14) apresentam densidades elevadas (10.92-19.80%), refletindo a maior variabilidade nestes movimentos complexos.
- O giroscópio mostra padrão inverso com atividades estáticas (A1-A3) que atingem 6.65-9.5%. Continua-se a verificar um aumento de outliers em atividades de transição neste caso de A8-A11 (7.47-11.87%)
- O magnetômetro mantém densidades geralmente baixas (0-4.7%) exceto, mais uma vez, nas transições, nomeadamente nas A9,A11,A14 e A15 (6.67-7.35%).

Estes resultados confirmam que a densidade de outliers varia significativamente consoante o sensor e tipo de atividade, com valores entre 0-20% que refletem as características específicas de cada movimento. As atividades de transição mostram sistematicamente maior variabilidade.

Os resultados do giroscópio ao início parecem contra intuitivos mas após pesquisar melhor a explicação encontrada deve-se a este medir velocidade angular, ou seja, rotação. Assim, durante atividades dinâmicas (como a atividade 14), o corpo roda com padrões regulares e suaves, as curvas de rotação são limpas e coerentes, logo existem menos outliers comparadamente aos outros sensores.

Pelo contrário, em atividades estáticas, o giroscópio deveria medir quase zero, mas há micro-oscilações involuntárias (respiração, pequenos ajustes posturais, tremores), e há ruído do sensor que se torna dominante, porque o sinal útil é muito pequeno. Desta forma explica-se a maior densidade de outliers nestas atividades.

Em baixo, estão representados graficamente o *boxplot* dos 3 sensores de cada dispositivo, divididos por cada atividade do *dataset*.

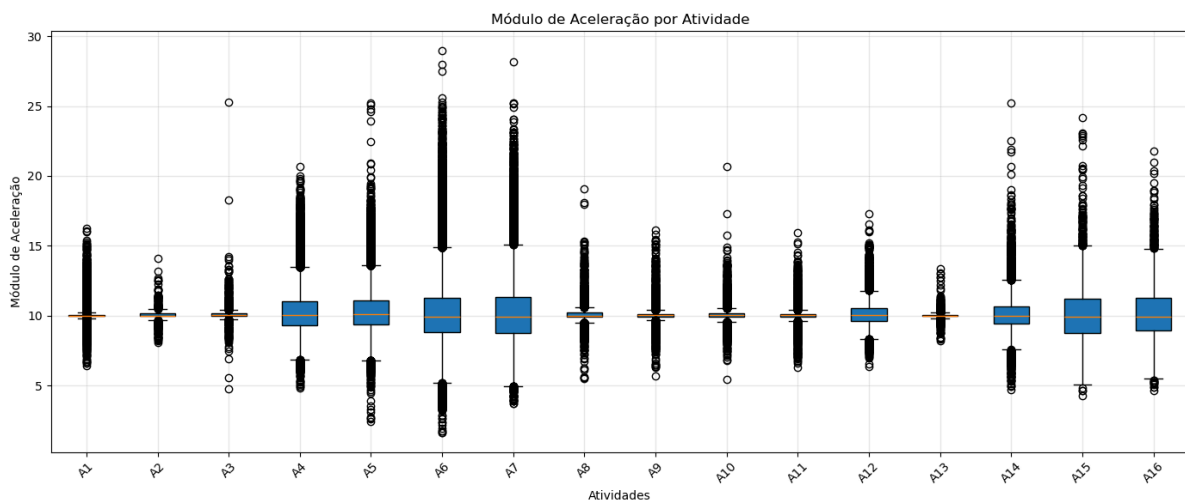


Fig 1. Boxplot do Módulo de Aceleração por Atividade

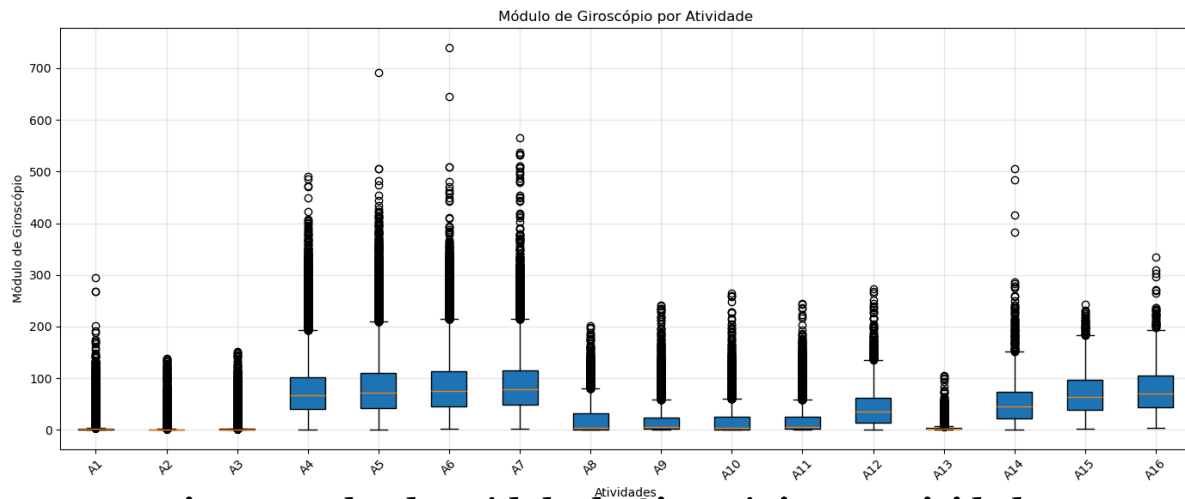


Fig 2. Boxplot do Módulo de Giroscópio por Atividade

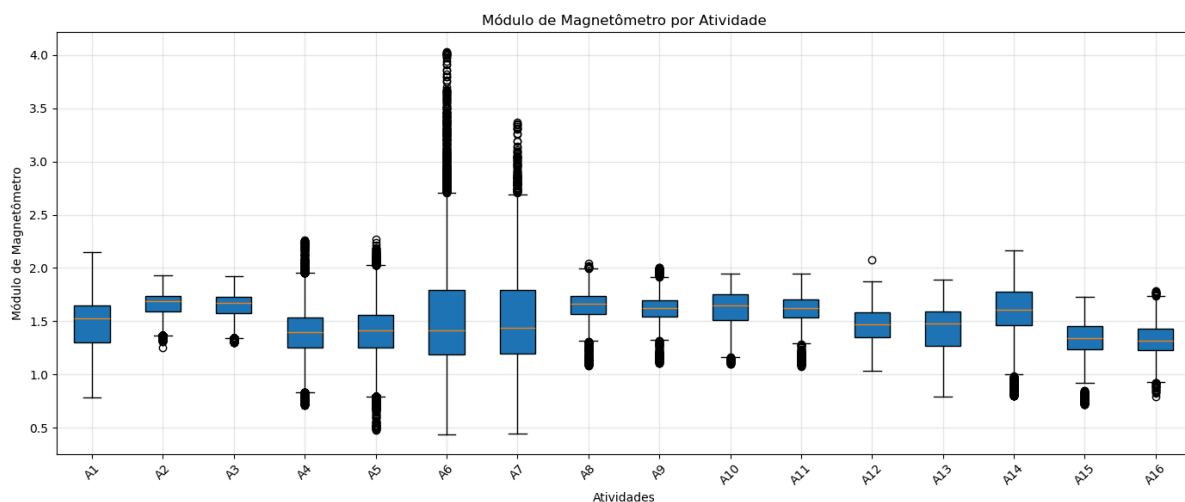


Fig 3. Boxplot do Módulo de Magnetômetro por Atividade

3.3 e 3.4

Nesta etapa foi criada uma função (**zscore**) para calcular os *z-scores* de um dado *array* e um *k-value*, de acordo com os resultados desta função, são então representados graficamente os pontos dentro do esperado a azul e os outliers a vermelho, de acordo com cada atividade através de outra função (**plot_outliers_zScore**), que chama a antes referida para obter os valores.

Aplicando este método com k igual a 3, 3.5 e 4, chegou-se às seguintes conclusões:

- O sensor da aceleração é o mais sensível às variações de movimento. As atividades estáticas apresentam percentagens baixas de outliers (<1%), enquanto as dinâmicas mantêm valores ligeiramente superiores (até 1,36% para k=3). Mudanças de postura ou movimentos combinados com fala

registram consistentemente os maiores valores (até 2,99%), refletindo a maior irregularidade destes movimentos. À medida que k aumenta, os outliers diminuem, mostrando menor sensibilidade a pequenas variações.

- O giroscópio apresenta menos outliers nas atividades dinâmicas, onde os movimentos são suaves e regulares, ligeiramente mais nas estáticas, devido ao ruído do sensor quando a rotação é mínima. Nas atividades de transição, o número de outliers aumenta, mas continua inferior ao da aceleração. Este comportamento evidencia que o giroscópio é sensível a mudanças de postura e pequenas oscilações, mas mantém coerência em movimentos contínuos.
- O sensor mais estável é sem dúvida o magnetômetro; mantém valores muito baixos de outliers em praticamente todas as atividades, mesmo nas de transição. Apenas em algumas transições rápidas ou movimentos combinados com fala se observam pequenas elevações pontuais.

Conclusão do uso do método z-score:

O método Z-Score, aplicado com $k=3$, 3.5 e 4 , mostra uma diminuição consistente da percentagem de outliers à medida que k aumenta, refletindo menor sensibilidade a pequenas variações.

No geral, $k=3.5$ apresenta um bom equilíbrio entre detecção de anomalias e robustez. A aceleração destaca-se na detecção de movimentos abruptos, o giroscópio evidencia ruído nas atividades estáticas, e o magnetômetro mantém comportamento consistente, sendo pouco sensível às mudanças de atividade.

Em baixo, está representado graficamente a nossa análise, com cada subplot com $k=3$, $k=3.5$ e $k=4$.

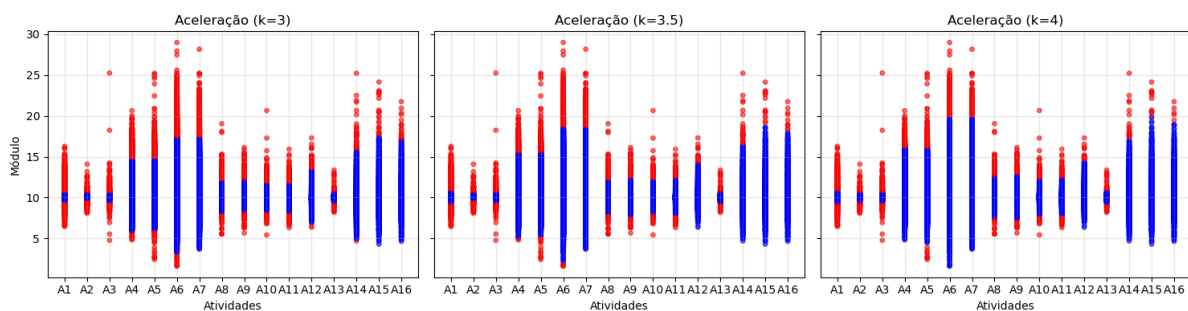


Fig 4. Representação do cálculo dos outliers do sensor de Aceleração segundo o método z-score

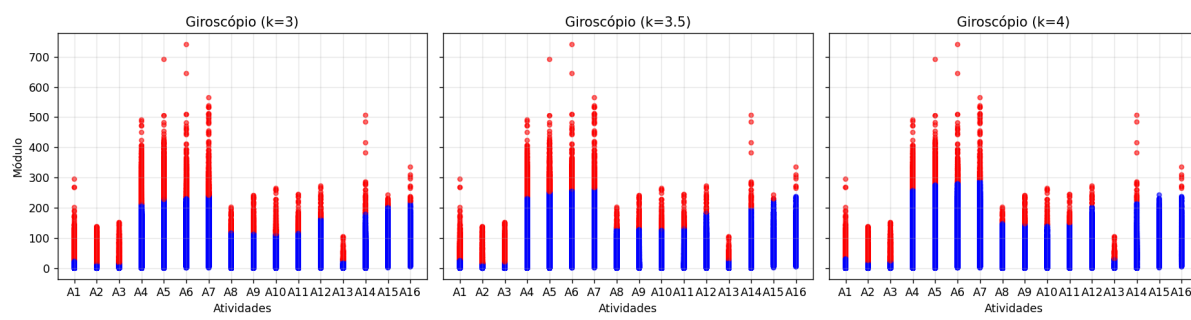


Fig 5. Representação do cálculo dos outliers do sensor de Giroscópio segundo o método z-score

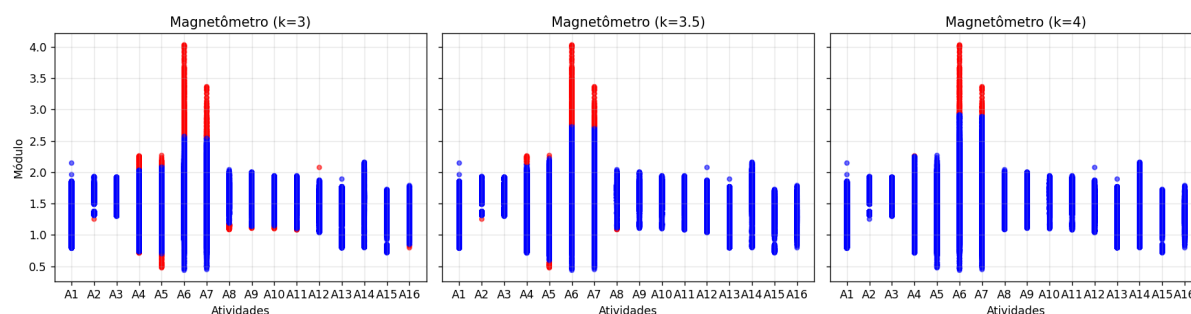


Fig 6. Representação do cálculo dos outliers do sensor de Magnetômetro segundo o método z-score

3.5

Comparações entre os métodos:

Verificaram-se diferenças nos três sensores indicados, sendo as seguintes as mais perceptíveis:

Aceleração: O IQR é mais sensível a variações abruptas, especialmente em transições. O Z-Score, com base em desvio-padrão global, é mais conservador e menos reativo a pequenas flutuações.

Giroscópio: Ambos os métodos revelam o mesmo comportamento: mais outliers em atividades estáticas. O Z-Score tende a suavizar os extremos, enquanto o IQR realça ruído quando o movimento é mínimo.

Magnetômetro: Ambos indicam que o magnetômetro é o sensor mais estável. O IQR ainda deteta ligeiros desvios nas transições; o Z-Score praticamente ignora-os, refletindo a sua robustez a pequenas oscilações.

O método IQR (Boxplot) revelou maior densidade média de outliers ($\approx 5 - 20 \%$), evidenciando melhor a variabilidade intra-atividade, especialmente no sensor da aceleração.

O Z-Score mostrou densidades inferiores ($\approx 0 - 3 \%$), refletindo maior estabilidade e filtragem de ruído.

Em suma, o IQR é mais exploratório e deteta variações locais, enquanto que o Z-Score é mais robusto e estatisticamente rigoroso, complementando-se na caracterização do comportamento dos sensores.

3.6 e 3.7

Para o K-means foi criado um ficheiro *k_means.py* que contém 3 funções principais:

- ***k_means_manual*** : Função principal que calcula o k_means sem recurso a bibliotecas exteriores. Esta função tem um número máximo de iterações padrão (100).
 - ***detect_outliers_kmeans*** : Função que calcula os outliers do k-means com um threshold padrão (2), ou seja, o número de desvios padrões acima da média a usar como limite para considerar um outlier.
 - ***plot_kmeans_results_3d*** : Função que faz os gráficos 3d.
 - ***print_outlier_density_per_activity*** : Função print de densidades de outliers por atividade para fins de comparação.
-

Os resultados com o k-means, com 4 clusters, são os seguintes:

- **Atividades estáticas (A1–A3):**

Praticamente sem outliers ($\leq 0.01\%$). Isto indica grande estabilidade e baixa variabilidade nos módulos dos sensores, coerente com posições fixas (sentado, de pé, a falar).

- **Atividades dinâmicas (A4–A7):**

Maiores densidades que no anterior, sobretudo em:

- A6 (14.02%) e A7 (15.38%), subir/descer escadas, com ou sem fala, são as mais irregulares, exibindo variação simultânea nos três sensores.
 - A4 (2.97%) e A5 (4.10%), caminhar simples e com fala mantêm moderada variabilidade, coerente com movimento rítmico mas sujeito a pequenas flutuações.
-

- **Atividades de transição (A8–A16)**

Densidades mistas:

- Transições simples (A8–A13) → $\leq 1\%$.
- Transições complexas ou compostas (A14–A16) → densidades elevadas (5–8%), devido a mudanças bruscas de movimento e aceleração.

Em baixo, temos uma representação gráfica do nosso método de cálculo de clusters através do método k-means, com 4 clusters por atividade.

Após experimentar com vários valores de k, chegamos à conclusão de que 4 clusters é o número ideal para realizar esta análise.

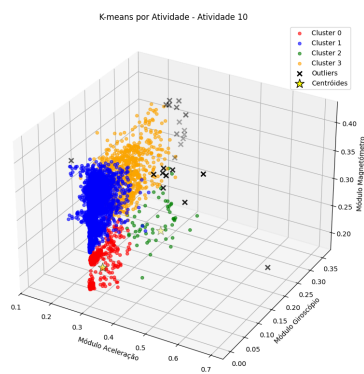


Fig 7. Atividade 10 representada por K-means.

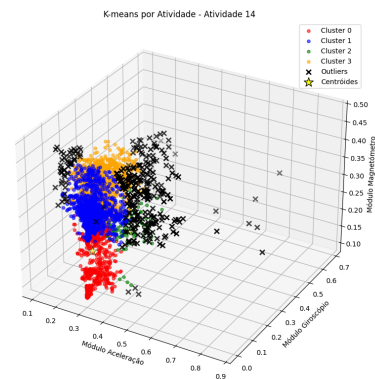


Fig 8. Atividade 14 representada por K-means.

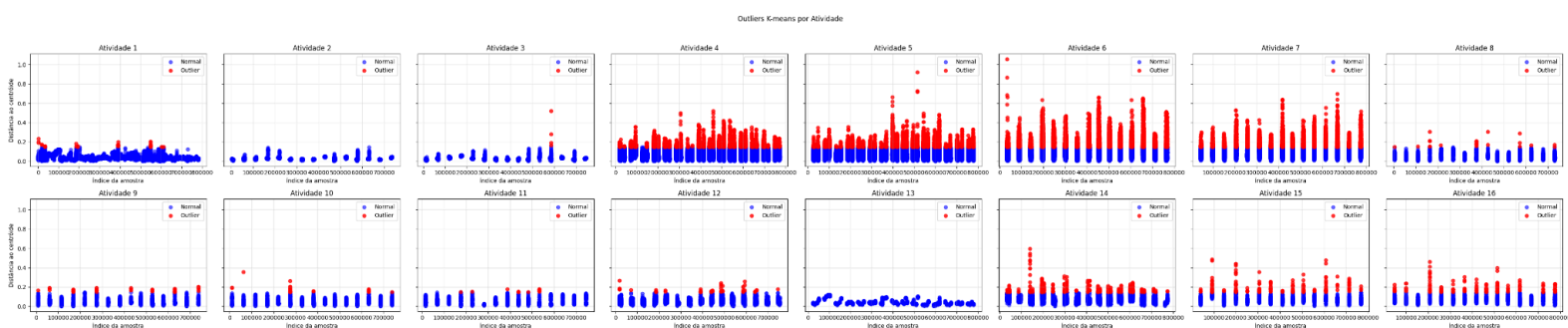


Fig 9. Representação gráfica das densidades de outliers pelo método K-means

Comparação com o método IQR (Boxplot):

O método IQR mostrou densidades muito maiores, chegando a 10–20% nas atividades de transição.

Já o K-means atinge valores semelhantes apenas nas atividades mais dinâmicas (A6, A7).

Isto confirma que K-means é mais conservador na detecção de outliers, identificando apenas desvios realmente afastados dos centroides médios, enquanto o IQR reage mais a dispersões locais.

Comparação com o método z-score:

O método do Z-score apresentou densidades de outliers baixas, inferiores a 1% nas atividades estáticas, cerca de 1,36% nas dinâmicas e até 2,99% nas de transição.

Já o K-means mostrou valores bastante superiores nas atividades com maior variabilidade, atingindo 5% a 15% nas dinâmicas, enquanto as estáticas mantiveram praticamente ausência de outliers.

Esta diferença resulta da natureza multivariada do K-means, que avalia conjuntamente aceleração, giroscópio e magnetômetro, captando variações combinadas e movimentos complexos. Já o Z-score, sendo univariável, apenas deteta desvios individuais, o que explica a sua menor sensibilidade.

3.7.1

Para o DBSCAN foi criado um ficheiro `dbscan.py` com duas funções principais:

- **dbscan_cluster:**
Função que aplica o algoritmo DBSCAN a um subconjunto de dados filtrado por atividades específicas. Utiliza os parâmetros ***eps*** (distância máxima entre pontos) e ***min_samples*** (mínimo de pontos por cluster), a implementação escolhida foi a da biblioteca `scikit-learn`. Retorna os dados filtrados, os rótulos de clusters e identifica outliers (label = -1). Apresenta o número de clusters e de outliers detetados no terminal, separados por atividade.
 - **plot_dbscan_results_3d:**
Função que gera gráficos 3D dos resultados do DBSCAN por atividade, usando os módulos da aceleração, giroscópio e magnetômetro como eixos. Cada cluster é colorido de forma distinta e os outliers são marcados a preto, permitindo análise visual da separação e densidade dos dados.
-

Por motivos de performance, só nos foi possível realizar a análise do DBSCAN para 3 participantes, utilizando **eps = 0.035** e **min_samples = 20**, obtivemos os seguintes resultados:

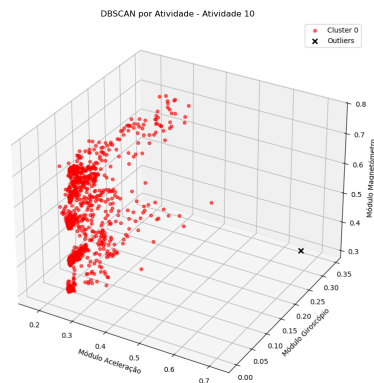


Fig 10. Atividade 10 representada por DBSCAN

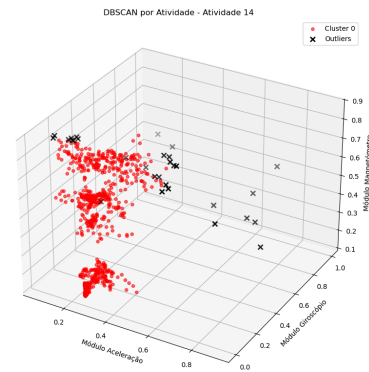


Fig 11. Atividade 14 representada por DBSCAN

Atividades estáticas (1–3):

o % de outliers, comportamento extremamente regular, sem ruído nem variação significativa.

Confirma a estabilidade esperada em posturas fixas (em pé, sentado, sentado a falar).

Atividades dinâmicas (4–7):

Pequenas percentagens (0.1 – 1.4 %), movimento consistente, mas com ligeira variação em alguns movimentos, nomeadamente os que envolvem escadas. As atividades 6 e 7 (subida/descida de escadas) apresentam mais irregularidade mecânica, logo mais outliers.

Atividades de transição (8–16):

Quase todas abaixo de 1 %, exceto A14–A16, com 2–3 % de outliers. Estas envolvem mudanças rápidas entre posturas, o que causa maior dispersão temporal e espacial dos dados, dando então origem a outliers.

Em baixo, uma representação gráfica (similar às anteriores) dos outliers detectados por este método.

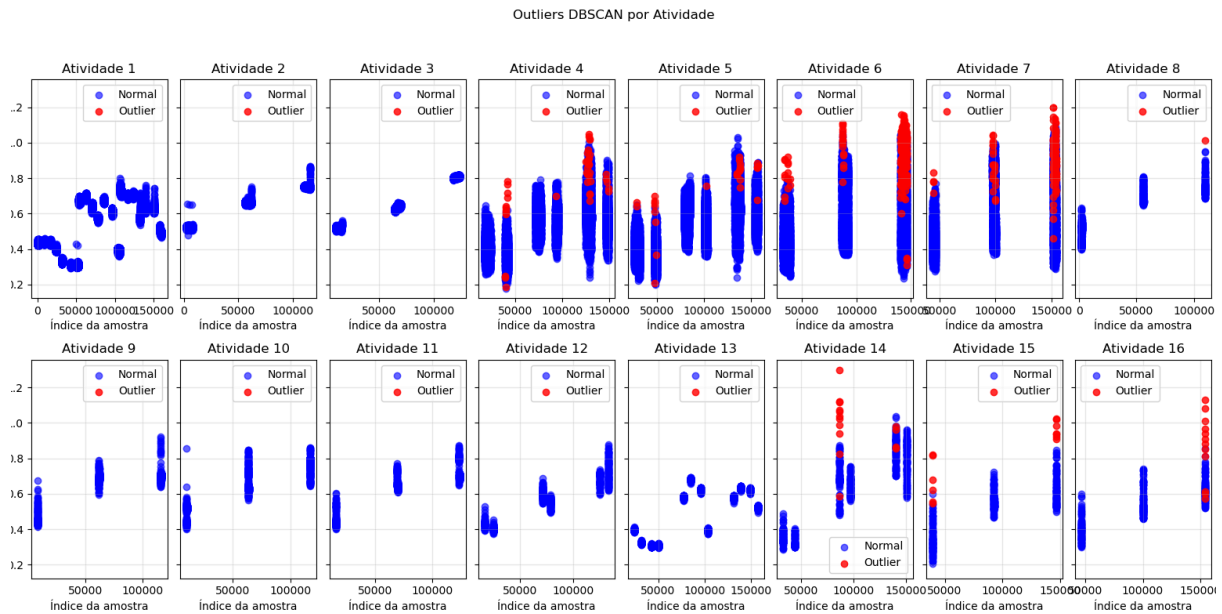


Fig 12. Representação gráfica dos outliers pelo método DBSCAN

4.1

A implementação deste ponto está presente no ficheiro *statistic_significance.py* onde, na função *analyze_statistical_significance*, é feita a análise estatística. Esta afere a normalidade da distribuição usando o teste Kolmogorov-Smirnov e de seguida aplica dois testes de significância estatística: ANOVA para dados normais e Kruskal-Wallis para dados não normais.

Após executar o código, verifica-se que os dados não apresentam normalidade ($p \approx 0.0000$ em todos os testes KS) e por isso analisamos os resultados do teste Kruskal-Wallis. Este apresenta valores extremamente significativos, onde o p-value (valor de significância estatística) indica probabilidade praticamente nula de que as diferenças entre atividades sejam devido ao acaso.

Os valores de H (estatística de Kruskal-Wallis) entre 4700-57400 confirmam diferenças muito significativas nas distribuições entre atividades, sendo particularmente elevados para o giroscópio. Isto deve-se ao elevado número de registos no dataset, o que o torna bastante sensível a pequenas variações entre grupos.

Concluindo, existe evidência estatística forte para afirmar que os módulos dos sensores permitem distinguir eficazmente as diferentes atividades humanas.

4.2

Após ler o artigo, foram identificadas as seguintes features:

Temporais:

- Média (Mean)
 - Variância (Variance)
 - Desvio Padrão (Standard Deviation)
 - RMS (Root Mean Square)
 - Máximo (Maximum)
 - Mínimo (Minimum)
 - Amplitude (Range = max - min)
 - Curtose (Kurtosis)
 - Assimetria (Skewness)
 - Entropia (Entropy)
 - Média das Diferenças Absolutas (Mean Absolute Deviation)
 - Área sob a Curva (Area Under Curve)
 - Energia do Sinal (Signal Energy)
-

Espetrais:

- Coeficientes FFT (Fast Fourier Transform coefficients)
 - Frequência Central (Spectral Centroid)
 - Largura de Banda Espectral (Spectral Bandwidth)
 - Planicidade Espectral (Spectral Flatness)
 - Fluxo Espectral (Spectral Flux)
 - Entropia Espectral (Spectral Entropy)
 - Energia em Bandas de Frequência (0-5Hz, 5-10Hz, 10-15Hz, 15-20Hz)
-

O código para extração destas features está presente no ficheiro *feature_extractor.py*. Neste começamos por calcular a taxa de amostragem para ser possível extrair as features espectrais e verificamos que esta é aproximadamente 50hz.

Depois de aproximarmos a taxa de amostragem, extraímos as features temporais e espectrais na classe *FeatureExtractor* com a ajuda dos métodos *temporal_features* e *spectral_features*. Para isto recorremos (como recomendado no enunciado) às bibliotecas Numpy, Scipy e às fórmulas presentes no artigo sugerido neste ponto.

Estes métodos de extração de features juntam-se num último método *extract_window_features* para considerar apenas uma janela temporal.

Por fim todas as janelas são extraídas na com a função *extract_features_4_2* e posteriormente gravadas num ficheiro para as features em si (features_x.csv) e outro para as labels (features_y.csv).

4.3

A função *pca_analysis* está presente no mesmo ficheiro das funções do ponto anterior e implementa a Análise de Componentes Principais (PCA) para redução de dimensionalidade. Inicia-se com a normalização das features, garantindo que todas as variáveis contribuem igualmente para o cálculo das componentes.

O PCA completo é aplicado para identificar as combinações lineares que maximizam a variância dos dados. Calcula-se a variância acumulada para determinar o número mínimo de componentes necessários para explicar 75% da variabilidade total (valor-alvo). Finalmente, o PCA é reaplicado com esse número ótimo de componentes, transformando os dados originais num espaço dimensional reduzido enquanto preserva a informação essencial. O processo reduz significativamente a dimensionalidade mantendo a representatividade dos dados.

Os resultados mostram um número de features reduzido de 252 para 8 que explica 75.16% do feature set. Estas são guardadas no ficheiro *features_X_pca.csv*.

4.4.1

Para obter as features comprimidas de um instante específico, aplica-se a mesma transformação usada durante o treino do PCA. Primeiro, normaliza-se o vetor de features do instante usando o scaler original.

De seguida, aplica-se a transformação PCA com o modelo treinado (*pca_final*), que projeta as features normalizadas no espaço reduzido de componentes principais. Por exemplo, para uma janela temporal específica com 240 features originais, obtém-se um vetor comprimido com apenas as componentes necessárias para explicar 75% da variância, tipicamente por volta das 10 componentes.

4.4.2

A principal vantagem do PCA é a significativa redução de dimensionalidade, que acelera os algoritmos subsequentes e reduz o overfitting. Além disso, remove correlações entre features e facilita a visualização de dados de alta dimensão.

No entanto, as componentes principais são combinações lineares de todas as features originais, perdendo-se a interpretabilidade individual. O PCA também assume relações lineares entre variáveis e pode descartar informação discriminativa relevante para a classificação, especialmente em dados com estruturas não-lineares complexas.

4.5

Fisher Score e ReliefF

Para a **seleção de features** foi criado o ficheiro **feature_selection.py**, que contém três funções principais:

fisher_score_selection:

Função que utiliza o **Fisher Score (ANOVA F-test)** para avaliar a relevância de cada feature em relação às classes. Ordena as variáveis pelos valores F e retorna os índices das *top_n* features mais discriminantes.

- **relieff_selection:**

Implementa o algoritmo **ReliefF**, que mede a importância das features com base na diferença entre amostras vizinhas de classes distintas. Permite definir o número de vizinhos e o número de features a selecionar, retornando as mais relevantes e os respectivos pesos de importância.

4.6

Identificação das 10 melhores features

Foram aplicados os métodos **Fisher Score** e **ReliefF** para selecionar as 10 *features* mais relevantes. O Fisher Score, baseado no ANOVA F-test, identifica variáveis com maior separação média entre classes, enquanto o ReliefF avalia a importância das *features* considerando vizinhança e interações locais.

Neste caso, não foram encontradas features em comum entre as *features* selecionadas. Isto deve-se às diferentes abordagens dos métodos.

4.6.1

Em baixo, encontram-se o top 10 features observadas em cada método:

FISHER SCORE	Peso	RELIEFF	Peso
accel_axis1_energy	20747.145955	mag_axis0_mean	0.518129
accel_axis1_rms	9.109807	mag_axis0_auc	129.036280
mag_axis0_spectral_c centroid	6.199200	mag_axis0_rms	0.521157
mag_axis2_spectral_c centroid	5.707574	accel_axis2_auc	784.896000
accel_axis1_auc	2267.985400	accel_axis2_rms	3.154878
mag_axis1_spectral_c centroid	9.959970	mag_axis2_energy	437.174413
mag_axis1_entropy	148.194343	accel_axis2_energy	2488.313893
mag_mag_spectral_c centroid	8.275354	accel_axis1_energy	20747.145955
mag_axis2_entropy	135.474793	mag_axis1_auc	169.767820

gyro_mag_rms	4.956453	accel_axis1_auc	2267.985400
--------------	----------	-----------------	-------------

4.6.2

Vantagens e Limitações

A abordagem reduz a dimensionalidade, melhora a interpretabilidade e diminui o risco de *overfitting*. Contudo, o Fisher Score analisa as variáveis de forma independente, ignorando correlações, enquanto o ReliefF é mais sensível ao ruído e à escolha dos vizinhos. Isto explica o porquê dos top 10 serem tão diferentes. Assim, ambos os métodos são complementares e devem ser avaliados em conjunto com o desempenho dos modelos subsequentes.

Conclusão

Concluindo, o trabalho permitiu desenvolver uma pipeline completa de análise e classificação de atividades humanas, desde a detecção de outliers até à extração e seleção de features. Os resultados obtidos demonstraram que os sensores permitem distinguir eficazmente diferentes tipos de movimento, validando a importância das fases de pré-processamento, redução de dimensionalidade e seleção de características.
